P2013R0
Freestanding Language: Optional `::operator new`

Published Proposal, 2020-01-10

This version:
https://wg21.link/P2013R0

Author:
Ben Craig (National Instruments)

Audience:
SG14, EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/ben-craig/freestanding_proposal/blob/master/core/new_delete.bs

---

Abstract

In freestanding implementations, standardize existing practices and make the default allocating `::operator new`s optional.

Table of Contents

§ 1. Revision History

R0 of this paper was extracted from [P1105R1].

The proposed solution is different than the one proposed for P1105R1, but the motivation is the same. The solution from P1105R1 is still listed as a design alternative.

§ 2. What is changing

On freestanding systems without default heap storage, the presence of the replaceable allocation functions (i.e. allocating `::operator new`, including the `nothrow_t` and `align_val_t` overloads, single and array forms) will be implementation defined. Implementations shall provide all

of the replaceable allocation functions, or none of them. If none of the replaceable allocation functions are provided, then an ODR-use of the functions will cause the program to be ill-formed. This will typically manifest as a linker error.

As a consequence of the above, coroutines that are relying on the global allocation functions will be ill-formed so long as those global allocation functions are not present.

No other core language features require `::operator new`. basic.stc.dynamic.allocation

`::operator delete` will be implementable as a no-op function on implementations that do not provide a default `::operator new`.

§ 3. What is staying the same

The replaceable deallocating `::operator delete` functions are still required to be present. `virtual` destructors ODR-use their associated `operator delete`(basic.def.odr), so keeping the global `::operator delete` allows those `virtual` destructors to continue building. Alternatives to this choice are discussed in § 7 Design Alternatives.

Calling `::operator delete` on a non-null pointer that did not come from `::operator new` is still undefined behavior new.delete.single new.delete.array. Calling `delete` on an object or base that didn't come from `new` is still undefined behavior expr.delete. This is what makes a no-op `::operator delete` a valid strategy on implementations without a global `::operator new`.

The replaceable allocation functions will still be implicitly declared at global scope in each translation unit basic.stc.dynamic. Non-ODR-uses of the replaceable allocation functions are still permitted (e.g. inside of uninstantiated templates). Implementations of the replaceable allocation functions can be performed by linking in an extra translation-unit with the definitions of the functions. Since this replacement typically happens at link time, ODR-uses of missing replaceable allocation functions usually won't be diagnosable at compile time.

Hosted implementations are unchanged. Users of freestanding implementations can still provide implementations of the replaceable allocation and deallocation functions. The behavior of `virtual` destructors is unchanged. The behavior of class specific `operator new` and `operator delete` overloads is unchanged. Non-allocating placement `::operator new` and `::operator delete` are still required to be present. The requirements on user-provided `::operator new` and `::operator delete` overloads remains the same, particularly those requirements involving error behaviors. Coroutines will behave the same so long as promise-specific allocators are used. The storage for exception objects will remain unspecified.

4. Why?

4.1. No allocations allowed

In space constrained and/or real-time environments, there is often no free store. These environments often cannot tolerate the space overhead for the free store, or the non-determinism from using the free store. In these environments, it is a desirable property for accidental global `new` usage to cause a build failure. With this proposal, users could expect a linker error when global `new` is used inappropriately.

FreeRTOS allows for both static and dynamic allocation of OS constructs [FreeRTOS_StaticVDynamic]. Static allocation in conjunction with a missing `::operator new` can help avoid overhead and eliminate accidental usage.

THREADX [THREADX] does not consider dynamic allocation a core service, and can be built without support for dynamic allocation in order to reduce application size. THREADX also distinguishes between byte allocation (general purpose) vs. block allocation (no-fragmentation elements of fixed size in a pool).

Also, by allowing a no-op `::operator delete` implementation, these space constrained applications can save code-size. No code needs to be present for `::operator delete` synchronization, free block coalescing, or free block searching.

4.2. No right way to allocate memory

In some target environments, there is no "right" way to allocate memory. In kernel and embedded domains, the implementer of the C++ toolchain doesn’t always know the "right" way to allocate memory on the target environment. This makes it difficult to provide an implementation for `::operator new`. The implementer cannot even rely on the presence of `malloc`, as it runs into the same fundamental problems.

As an example, in the Microsoft Windows kernel environment, there are two leading choices about where to get dynamic memory [MSPools]. Users can get memory from the non-paged pool, which is a safe, but scarce resource; or users can get memory from the paged pool, which is plentiful, but not accessible in many common kernel operations. Non-paged pool must be used any time the allocated memory needs to be accessible from an interrupt or from a "high IRQL" context. The author has had experience with both paged pool and non-paged pool as defaults, with the predictable outcome of crashes with paged pool defaults and OOM with non-paged pool defaults. The implementer of the C++ toolchain is not in a good position to make this choice for the user.

In the Linux kernel environment, `kmalloc` [kmalloc] with the `GFP_KERNEL` should be used when allocating memory within the context of a process and outside of a lock, but `GFP_ATOMIC` should be used when allocating memory outside the context of a process, such as inside of an interrupt. The implementers of the C++ runtime are in no position to know which is the correct flag to use by default. Using `GFP_KERNEL` when `GFP_ATOMIC` is needed will result in crashes from interrupt code and deadlocks. Using `GFP_ATOMIC` when `GFP_KERNEL` is appropriate will result in reduced system performance, spurious OOM errors, and premature exhaustion of emergency memory pools.

Freestanding implementations are intended to run without the benefit of an operating system (basic.def.odr). However, the name of the function that supplies dynamic memory is usually an OS-specific detail. The C++ implementation should not (and may not) know the name of the function to request memory. The Windows kernel uses `ExAllocatePoolWithTag`. In the Linux kernel, `kmalloc` is the main function to use. In FreeBSD, a function named `malloc` is present, but it takes different arguments than the C standard library function of the same name. FreeRTOS uses `pvPortMalloc`, and THREADX uses `tx_byte_allocate`. Home-grown OSes will likely have other spellings for memory allocation routines.

Today's C++ implementations don't provide `::operator new` implementations for all possible targets. Doing so isn't a plausible goal, especially when the home-grown OSes are taken into account. This means that users are already forced into choosing between not having `::operator new` support and providing their own implementation. We should acknowledge and standardize this existing practice, especially since we already have the extension point mechanism in place.

§ 4.3. What about allocators?

The C++20 freestanding freestanding library does not include allocators. [P1642R1] proposes adding allocator machinery to freestanding, but doesn't add `std::allocator` itself. In addition, none of the allocating standard containers are in C++20's freestanding library or any current freestanding library proposal that the author is aware of. From a minimalist freestanding perspective, allocators aren't a solution.

Allocators are still useful in a less-than-minimal freestanding implementation. In environments with dynamic memory, custom allocators can be written and used with standard containers, assuming that the containers are present in the implementation. This could be done even if a global `::operator new` is not present. The author has used `stlport::vector<int, PagedLockedAllocator>` successfully in these environments.

`std::allocator` is implemented in terms of global `::operator new`. In practice, it would be easy for an implementation to have an implementation of `std::allocator` in a header / module, and

have that header still compile just fine, as it wouldn't ODR-use `::operator new` until it was instantiated. If the user has provided a global `::operator new`, then `std::allocator` would have the same semantics as mandated by the standard. If the global `::operator new` is not present, then uses of `std::allocator` would fail to link, which would still be conforming behavior on a freestanding implementation.

Some facilities in the standard library (e.g. `make_unique`) are implemented in terms of `new`, and not an allocator interface. It is useful to make these facilities error when dynamic memory isn't available, and it is also useful to be able to control which memory pool is used by default.

§ 4.4. `virtual` destructors

A no-op `::operator delete` is still provided in order to satisfy `virtual` destructors. `virtual` destructors ODR-use their associated `operator delete`(basic.def.odr). This approach has the disadvantage that there is a small, one-time overhead for the first `virtual` destructor in a program, even if there are no usages of `new` or `delete`. The overhead is small though, and you only pay for the overhead if you use `virtual` destructors.

Ideally, if neither `new` nor `delete` is ever called, we wouldn't need an `operator delete`. This proposal still requires some `operator delete` to exist, though that `operator delete` can be a no-op.

§ 4.5. Likely misuses and abuses

Users are likely to provide overloads of `::operator new` that do not follow the requirements set forth in new.delete, particularly the requirements around throwing `bad_alloc`. Ignoring this requirement will still result in undefined behavior, just as it does in C++20. Some compilers optimize assuming that the throwing forms of `new` will never return a null pointer [throwing_new]. A likely outcome of the undefined behavior is unexpectedly eliding null checks in the program source. This problem already exists today, and this proposal makes it no worse.

§ 5. Experience

The proposed design has field experience in a micro-controller environment. GCC was used, and the language support library was intentionally omitted. A no-op `::operator delete` was provided by the users. The no-op `::operator delete` enabled a small amount of code sharing between a hosted

environment and this micro-controller environment. Some of the shared code involved classes with `virtual` destructors.

§ 6. Polling history

Jan 8, 2020 SG14 Telecon:

Forward P2013 as is with the minor editing quotes

SF/F/N/A/SA

9/10/0/0/0

approves to go to EWG

§ 7. Design Alternatives

§ 7.1. Alternative 0: All-or-nothing allocating `::operator new`, no-op default deallocation functions (Proposed above)

This option preserves much functionality, without using any novel techniques. See above for further explanation.

§ 7.2. Alternative 1: Optional throwing `::operator new`s, no-op default deallocation functions

Rather than making all of the replaceable allocation functions optional, we could make just the throwing `::operator new`s optional (array and single form, with and without `align_val_t` parameters). The library would still be required to provide the `nothrow_t` overloads.

The `nothrow_t` overloads are specified to forward to an appropriate throwing overload. That implementation would still be fine on a system without dynamic storage available. This alternative was not selected as it is more difficult to teach, and because the target audience would likely be astonished that the `nothrow_t` overload has a `try`/`catch` in it.

7.3. Alternative 2: No deallocation functions

The presence of the replaceable deallocation functions is implementation defined. `virtual` destructors will be ill-formed unless the implementation provides the deallocation function, the user provides a global `::operator delete` function, or the user provides a class specific `operator delete` overload.

This alternative has the benefit of being zero overhead and very explicit, but it has troublesome consequences for implementations. There are several language support classes that have `virtual` destructors, and something would need to be decided for them. Notably, `type_info` and the `exception` hierarchy all have `virtual` destructors. The standard library implementers may be prohibited from providing `operator new` and `operator delete` overloads (conforming#member.functions). Alternatively, the facilities that require classes with `virtual` destructors could all be off-limits until `operator delete` was made available. This would eliminate many cases with exceptions, `dynamic_cast` on references, and `typeid`.

If we were to adopt this alternative, many users would provide a no-op `::operator delete` in their code, giving their code the same semantics and trade-offs as the proposed solution.

7.3.1. Experience

This alternative has field experience. MSVC's /kernel [kernel_switch] flag omits definitions for `::operator new` and `::operator delete`. Users of Clang and GCC can choose to not link against the language support library, and therefore not have `::operator new` and `::operator delete` support, as well as many other language support features.

7.4. Alternative 3: No deallocation functions and new ODR-used rules for virtual destructors

The presence of the replaceable deallocation functions is implementation defined. Change `virtual` destructors so that they generate a partial vtable and don't ODR-use `::operator delete`. Make `new` expressions ODR-use `::operator delete` and complete the vtable.

7.4.1. How could this virtual destructor ODR-use change be implemented?

First, this is only a problem that needs to be solved on systems without a default heap. This means that typical user-mode desktop and server implementations would be unaffected.

Existing linkers already have the ability to take multiple identical virtual table implementations and pick one for use in the final binary. A potential implementation strategy is for compilers and linkers to support a new "weaker" linkage. When the default heap is disabled, the compiler would emit a vtable with a `nullptr` or pure virtual function in the virtual destructor slot. When `new` is called, a "stronger" linkage vtable would be emitted that has the deleting destructor in the virtual destructor slot. The linker would then select a vtable with the strongest linkage available. Today's linkage would be considered "stronger". Only partially filled vtables would have "weaker" linkage.

7.4.2. ABI impact

Mixing multiple object files into the same program should be fine, even if some of them have a default heap and some don't. All the regular / "strong" linkage vtables should be identical, and all the "weaker" linkage vtables should be identical. If anyone in the program calls any form of `new`, the deleting destructor will be present and in the right slot. If no-one calls `new` in the program, then no-one should be calling `delete`, and the empty vtable slot won't be a problem.

Shared libraries are trickier. Vtables aren't always emitted into every translation unit. Take shared library "leaf" that has a default heap. It depends upon shared library "root" that does not have a default heap. If a class with a virtual destructor is defined in "root", along with its "key function", then a call to `new` on the class in "leaf" will generate an object with a partial vtable. Calling `delete` on that object will cause UB (usually crashes).

Lack of a default heap should generally be considered a trait of the platform. Mixing this configuration shouldn't be a common occurrence.

7.4.3. Experience

This alternative is novel, and does not have implementation or usage experience.

8. Acknowledgments

Thank you to the many reviewers of this paper: Brandon Streiff, Irwan Djajadi, Joshua Cannon, Brad Keryan, Alfred Bratterud, Phil Hindman, Arthur O'Dwyer, Laurin-Luis Lehning, JF Bastien, and Matthew Bentley

§ References

§ Informative References

[FreeRTOS_StaticVDynamic]

FreeRTOS Documentation. Static Vs Dynamic Memory Allocation. URL: https://www.freertos.org/Static_Vs_Dynamic_Memory_Allocation.html

[KERNEL_SWITCH]

Microsoft Documentation. /kernel (Create Kernel Mode Binary). URL: https://docs.microsoft.com/en-us/cpp/build/reference/kernel-create-kernel-mode-binary

[KMALLOC]

kernel.org. kmalloc. URL: https://www.kernel.org/doc/htmldocs/kernel-api/API-kmalloc.html

[MSPools]

Microsoft Documentation. POOL_TYPE enumeration. URL: https://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/ne-wdm-_pool_type

[P1105R1]

Ben Craig; Ben Saks. Leaving no room for a lower-level language: A C++ Subset. URL: https://wg21.link/P1105R1

[P1642R1]

Ben Craig. Freestanding Library: Easy [utilities], [ranges], and [iterators]. URL: https://wg21.link/P1642R1

[THREADX]

THREADX(R) RTOS - Royalty Free Real-Time Operating System. URL: https://rtos.com/solutions/threadx/real-time-operating-system/

[THROWING_NEW]

Microsoft Documentation. /Zc:throwingNew (Assume operator new throws). URL: https://docs.microsoft.com/en-us/cpp/build/reference/zc-throwingnew-assume-operator-new-throws?view=vs-2019